БроньЛаб

Этап 2: Архитектура и техническое задание

Цель этапа: перевести бизнес-гипотезу «БроньЛаб» в минимальную, однозначную техническую архитектуру и ТЗ, которое можно передать одному разработчику для реализации MVP за несколько рабочих дней.

---

1. Декомпозиция бизнес-требований

1.1. Аудит бизнес-гипотезы

Главная проблема: нет достоверного ответа, свободен ли нужный прибор в заданное время. Студенту нужна камера, проектор или лабораторный прибор к конкретному занятию. Он пишет ответственному, ждёт ответа и только затем узнаёт, что вещь уже занята. Ответственный вручную сверяет таблицу и сообщения в чате. Возможны две заявки на один период.

MVP должен одновременно решить три задачи:

- показать актуальную занятость оборудования;
- позволить забронировать свободный интервал;
- не допустить конфликтующих броней.

Функция MVP	**Что делает пользователь**	**Что принимает система**	**Обработка**	**Результат**
1. Бронирование	Выбирает прибор, дату и интервал в рабочие часы	ID прибора, дата, время начала и конца	Сервер проверяет пересечение с действующими бронями в транзакции	Бронь подтверждена; конфликтующие заявки невозможны
2. Каталог	Просматривает список доступных приборов	Список оборудования, внесённый	Показывает название, место получения,	Пользователь выбирает подходящее оборудование

Функция MVP	Что делает пользователь	Что принимает система	Обработка	Результат
		администратором	краткое описание	
3. Просмотр занятости	Открывает карточку прибора и смотрит занятые интервалы	ID прибора и выбранная дата	Формирует список занятых интервалов	Пользователь видит свободное время и может планировать работу

1.2. Ограничения и границы

- Приложение — закрытое веб-приложение для одной кафедры.
- MVP поддерживает бронирование в пределах одного рабочего дня.
- Не требуются: интеграция с вузовской учётной записью, SSO, электронная почта, мессенджеры, оплата, автоматический учёт физической выдачи.
- Физическая выдача оборудования остаётся за ответственным сотрудником.
- Администратор пилота заранее вносит оборудование и выдаёт учётные записи ограниченному кругу пользователей.
- Технология: адаптивный веб-интерфейс, небольшой backend с REST API, SQLite для пилота.
- Проверка пересечений выполняется в транзакции на сервере.
- Логи хранятся локально. Срок хранения логов MVP — 30 дней.
- Один справочник оборудования не должен превышать разумный лимит пилота: 10–30 единиц.
- Время должно быть согласовано в едином часовом поясе подразделения.
- Системные и скрытые данные не обрабатываются без явного указания администратора.

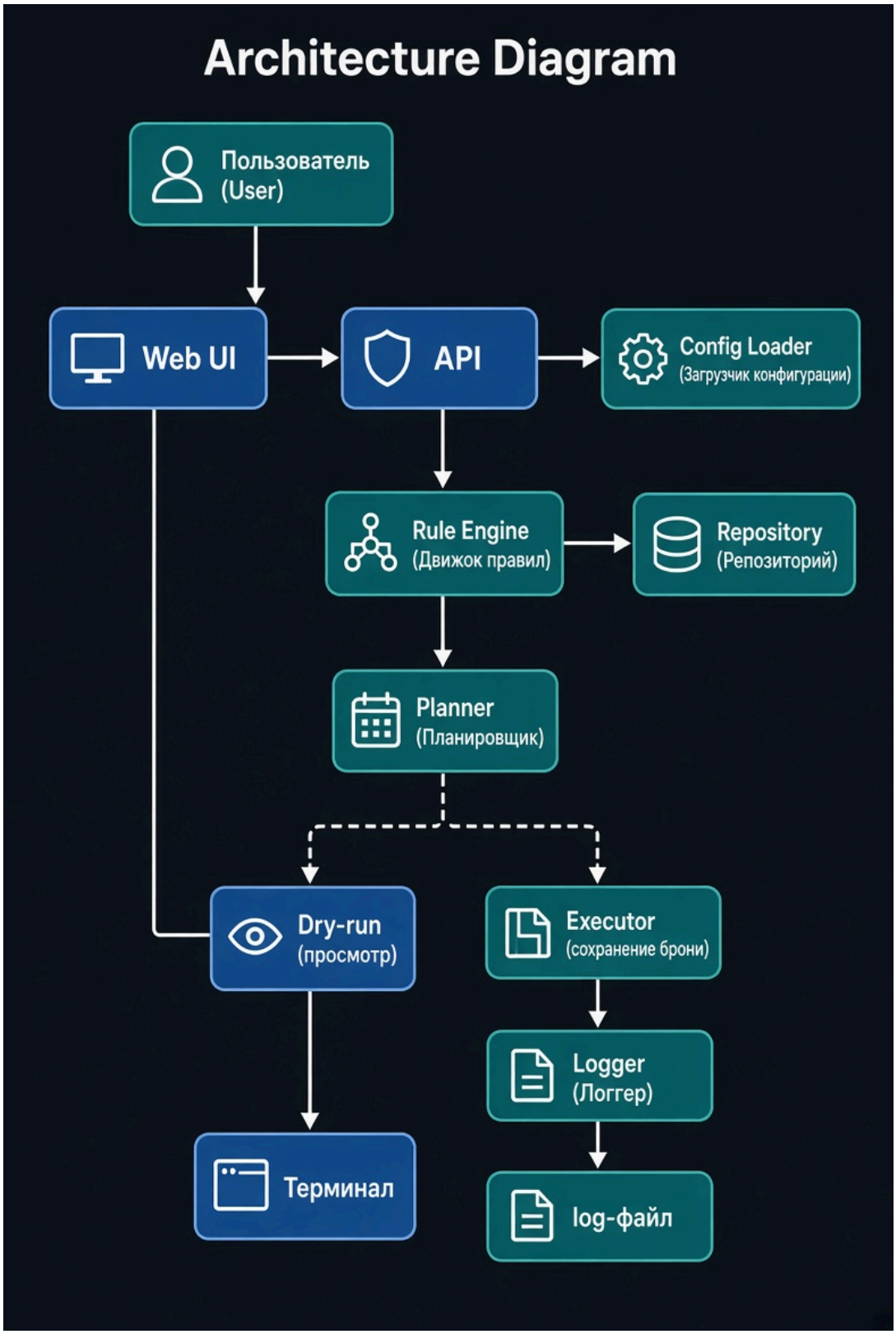
2. Проектирование архитектуры

2.1. Компонентный состав

Компонент	Ответственность	Технология
Web UI	Каталог, карточка прибора, форма брони, список занятости	HTML/CSS/JS, адаптивная вёрстка

Компонент	**Ответственность**	**Технология**
API	Принимает запросы от UI, возвращает данные каталога и броней	Python + Flask/FastAPI
Config Loader	Читает настройки пилота: рабочие часы, список приборов, администратор	Python + YAML
Repository	Получает список оборудования и броней из БД	SQLAlchemy / sqlite3
Rule Engine	Проверяет правила: рабочие часы, пересечение интервалов, доступность прибора	Обычные Python-функции
Planner	Формирует план операции: создать бронь, отклонить конфликт, показать занятость	Python-объекты/список
Executor	Сохраняет бронь в БД, обновляет занятость	SQLite + транзакции
Logger	Записывает результат каждой операции и ошибки	logging
Scheduler	Периодическая проверка или напоминание	cron ОС

2.2. Архитектурная схема



Data Flow:

1. Config Loader читает YAML и проверяет структуру. При ошибке выполнение прекращается до доступа к данным.
2. Repository получает список оборудования и действующих броней из SQLite.
3. Rule Engine проверяет правила: рабочие часы, пересечение интервалов, доступность прибора.
4. Planner формирует список операций: создать бронь, отклонить конфликт, показать занятость.
5. В режиме `--dry-run` Executor не изменяет БД, а только показывает план и пишет его в лог.
6. В обычном режиме Executor выполняет операции. Для брони используется транзакция; при конфликте бронь не создаётся.
7. Logger фиксирует успешные операции и ошибки.
8. cron при необходимости запускает ту же команду по расписанию.

---

3. Модель данных

Реляционная БД для MVP требуется, но в минимальном виде. Основные сущности представлены YAML-конфигурацией и таблицами SQLite.

Сущность	**Поля**	**Обязательность**	**Правила**
Config	version, work_hours, equipment, admin_users	version, work_hours	version = 1; рабочие часы должны быть валидными
Equipment	id, name, location, description, status	id, name, location	status = available / hidden
Booking	id, equipment_id, user_id, date, start_time, end_time, status	equipment_id, user_id, date, start_time, end_time	start_time < end_time; интервал в рабочих часах
User	id, name, role	id, name, role	role = student / teacher / admin

Сущность	Поля	Обязательность	Правила
Operation	source, destination, action, status, error	source, action, status	создаётся на время запуска и попадает в лог

Пример конфигурации

```
version: 1

work_hours:

  start: "09:00"

  end: "18:00"

equipment:

  - id: "camera-01"

    name: "Камера Sony A7"

    location: "Медиакласс, ауд. 312"

    description: "Полнокадровая камера для учебных проектов"

    status: "available"

  - id: "projector-01"

    name: "Проектор Epson"

    location: "Лаборатория, ауд. 210"

    description: "Переносной проектор для занятий"

    status: "available"

admin_users:

  - id: "admin-01"

    name: "Лаборант"
```

role: "admin"

Правила целостности

- `version` обязателен и должен быть равен 1.
- `work_hours.start` и `work_hours.end` должны быть валидными и start < end.
- `equipment.id` должен быть уникальным.
- `status` принимает только `available` или `hidden`.
- Нельзя бронировать прибор на интервал вне рабочих часов.
- Нельзя бронировать прибор на пересекающийся интервал.
- Если правило не подходит, бронь не создаётся.
- Один прибор не должен одновременно получать две брони на пересекающееся время; приоритет получает первая подтверждённая бронь.
- Физическая выдача остаётся за ответственным сотрудником и не входит в MVP.

---

4. Техническое задание

4.1. CLI-контракт

Команда	**Назначение**	**Параметры**	**Результат**
`bronylab run`	Запуск обработки	`--config PATH`, `--dry-run`	Код 0 при успехе, ненулевой при ошибке
`bronylab validate`	Проверка конфига без обработки	`--config PATH`	Сообщение о корректности или список ошибок
`bronylab --help`	Справка	—	Список доступных команд и параметров

Формат успешного результата:

```
OK: equipment=30, bookings=12, conflicts=0, skipped=5, errors=0
```

Формат ошибок:

Единый формат: `ERROR <CODE>: <сообщение>`

Код	Ситуация	Поведение
CONFIG_NOT_FOUND	Файл конфигурации отсутствует	Завершить запуск без операций
CONFIG_INVALID	YAML или обязательное поле некорректно	Завершить запуск без операций
EQUIPMENT_NOT_FOUND	Прибор не найден в каталоге	Не создавать бронь, записать ошибку
BOOKING_CONFLICT	Интервал пересекается с действующей бронью	Не создавать бронь, записать ошибку
OUTSIDE_WORK_HOURS	Интервал вне рабочих часов	Не создавать бронь, записать ошибку
PERMISSION_DENIED	Нет прав на операцию	Пропустить операцию, записать ошибку
DB_ERROR	Ошибка базы данных	Не считать операцию успешной, записать stderr
INVALID_RULE	Правило имеет неверный формат	Отклонить конфигурацию до запуска

Примеры валидных и невалидных данных

Пример	Статус	Причина
`start_time: "10:00"`	Валидно	Входит в рабочие часы
`end_time: "18:00"`	Валидно	Совпадает с концом рабочего дня
`start_time: "08:00"`	Невалидно	Вне рабочих часов
`end_time: "10:00"`	Невалидно	Конец раньше начала
`equipment_id: "camera-01"`	Валидно	Прибор существует
`equipment_id: "unknown-01"`	Невалидно	Прибора нет в каталоге

4.2. Алгоритмические процессы

Бронирование

1. Прочитать конфигурацию.
2. Получить список приборов и действующих броней.
3. Проверить, что выбранный интервал входит в рабочие часы.
4. Проверить, что интервал не пересекается с действующими бронями.
5. Создать бронь в транзакции.
6. Записать результат в лог.

Просмотр занятости

1. Для выбранного прибора определить дату.
2. Получить список занятых интервалов.
3. Если интервалов нет — показать пустое состояние.
4. Если интервалы есть — показать их пользователю.
5. Зафиксировать успешную или неуспешную операцию.

Параметры

- По умолчанию выполнение производится с dry-run при первом запуске не требуется автоматически; пользователь явно использует `--dry-run` для предварительного просмотра.
- Конфигурация и параметры не зашиваются в код.
- Константы: версия конфига, формат логов и стандартное имя лог-файла.
- Сложные правила MIME и регулярные выражения не входят в обязательный объём первой реализации, чтобы сохранить срок менее недели.

4.3. Интерфейс CLI и состояния

Состояние	**Что видит пользователь**
Старт	Название БроньЛаб, версия и сообщение о начале проверки каталога.
Загрузка каталога	Краткий статус: `Loading equipment...` и количество доступных приборов.
Dry-run	Список планируемых броней и итоговая статистика; база данных не изменяется.
Успех	Итоговая статистика: equipment, bookings, conflicts, skipped, errors.

Состояние	Что видит пользователь
Ошибка	Понятное сообщение с кодом ошибки и ID проблемного прибора или брони.
Пустое состояние	Сообщение `No bookings found for the selected equipment and date.`

Прототипы ключевых экранов

[Экран 1: запуск]

$ bronylab run --config config.yaml --dry-run

БроньЛаб 0.1

Loading equipment from config.yaml...

Found: 30 equipment units

PLAN

  BOOK camera-01 2026-10-06 10:00-12:00

  SKIP projector-01 2026-10-06 09:00-10:00

  CONFLICT camera-01 2026-10-06 11:00-13:00

Summary: equipment=30 bookings=12 conflicts=1 skipped=5 errors=0

DRY-RUN: no bookings were saved.

[Экран 2: обычный запуск]

$ bronylab run --config config.yaml

Executing...

OK: equipment=30 bookings=12 conflicts=0 skipped=5 errors=0

Log: ~/.bronylab/bronylab.log

[Экран 3: ошибка]

ERROR BOOKING_CONFLICT: camera-01 is already booked for 2026-10-06 11:00-13:00

No bookings were saved.

---

5. Планирование реализации (WBS)

План специально ограничен минимально необходимым объёмом. Цель — один разработчик и менее недели, без отдельного сервера, базы данных и сложного интерфейса.

№	Задача	Оценка	Зависимости	Результат дня
1	Создать Python-проект, CLI и окружение	2 ч	—	Команда запускается
2	YAML-конфиг + валидация	3 ч	1	Рабочий validate
3	Загрузка каталога оборудования и пользователей	3 ч	1	Список приборов и ролей
4	Rule Engine: рабочие часы, пересечение интервалов	4 ч	2, 3	Правила дают план брони
5	Dry-run и Planner	2 ч	4	Безопасный просмотр плана
6	Executor: создание и сохранение брони	4 ч	5	Бронь в SQLite
7	Просмотр занятости по прибору и дате	2 ч	6	Список занятых интервалов

№	Задача	Оценка	Зависимости	Результат дня
8	Логирование и обработка ошибок	3 ч	5–7	Понятные ошибки и лог
9	cron + README с примером	2 ч	8	Автоматический запуск
10	Тесты, проверка граничных случаев, исправления	5 ч	2–9	Готовый MVP

Итого: около 30 часов чистой разработки. Это соответствует примерно 4 рабочим дням по 7–8 часов и оставляет небольшой резерв на исправления в пределах одной недели.

Критический путь

Окружение → конфигурация → каталог оборудования → Rule Engine → Planner/dry-run → Executor → логирование/ошибки → тестирование. cron и README могут выполняться после основного Executor и не блокируют разработку ядра.

---

6. Граничные случаи

Случай	Ожидаемое поведение
Прибор не найден	Ошибка EQUIPMENT_NOT_FOUND; программа не падает с traceback.
Интервал вне рабочих часов	Ошибка OUTSIDE_WORK_HOURS; бронь не создаётся.
Интервал пересекается с действующей бронью	Ошибка BOOKING_CONFLICT; бронь не создаётся.
Одновременные заявки на один прибор	Серверная транзакция пропускает только первую; вторая получает BOOKING_CONFLICT.

Случай	**Ожидаемое поведение**
Нет прав	Операция пропускается, ошибка попадает в лог.
Прибор скрыт (status = hidden)	Прибор не показывается в каталоге и не доступен для брони.
Нет броней на выбранную дату	Успешный запуск с нулевой статистикой и пустым состоянием.
Ошибка базы данных	Операция не считается успешной; ошибка логируется.
Dry-run	Ни одна бронь не сохраняется в БД.
Запуск повторён	Уже созданные брони не дублируются.

---

7. Самопроверка соответствия этапу 1

Требование этапа 1	**Как закрывается в этапе 2**
Бронирование	Rule Engine + Planner + Executor; проверка пересечений в транзакции.
Каталог	Config Loader + Repository; карточки приборов с местом выдачи.
Просмотр занятости	Repository + Planner; список занятых интервалов по прибору и дате.
Защита от накладок	Транзакция и правило BOOKING_CONFLICT.
Работа без SSO и оплаты	Администратор выдаёт учётные записи; оплата не реализуется в MVP.
Логи	Python logging + локальный файл.
User Stories	Все три покрыты: бронирование, каталог, просмотр занятости.

---

8. Self-Check по чек-листу участника 2

- ✓ Функциональные требования извлечены и переведены в системные действия.
- ✓ MVP ограничен тремя функциями из этапа 1; лишняя серверная инфраструктура исключена.
- ✓ Нефункциональные требования, безопасность, приватность и хранение истории определены.
- ✓ Определены границы данных, форматы конфигурации и разумные лимиты MVP.
- ✓ Описаны клиентская часть, серверная логика, хранилище и вычислительные компоненты.
- ✓ Дана архитектурная схема и последовательность Data Flow.
- ✓ Определена модель данных и правила целостности.
- ✓ Описаны CLI-контракты, успешные ответы и единый формат ошибок.
- ✓ Описаны алгоритмы бронирования и просмотра занятости.
- ✓ Конфигурация вынесена в YAML.
- ✓ Описаны ключевые CLI-экраны и состояния: запуск, загрузка каталога, dry-run, успех, ошибка, пустой результат.
- ✓ WBS разбит на задачи с оценкой и зависимостями.
- ✓ Есть критический путь.
- ✓ Объём реалистичен для одного разработчика менее чем за неделю.

---

9. Передача участнику 3

Перед разработкой необходимо синхронно пройти три вещи:

- Показать архитектурную схему и объяснить путь: config → equipment → rules → plan → execute → log.
- Отдельно продемонстрировать dry-run как основной механизм безопасной проверки брони.
- Зафиксировать, что MVP не требует SSO, оплаты и автоматического учёта физической выдачи.
- Передать пример config.yaml и команды `validate`, `run --dry-run`, `run`.
- Передать критерии готовности: все три функции работают, конфликтующие брони невозможны, ошибки не приводят к потере данных, тесты проходят, cron запускает ту же команду.

---

10. Definition of Done для этапа 3

- Команда validate принимает корректный YAML и отклоняет некорректный.
- Каталог оборудования загружается и отображается в веб-интерфейсе.
- Бронирование свободного интервала работает.
- Пересекающиеся брони отклоняются с ошибкой BOOKING_CONFLICT.
- Просмотр занятости показывает занятые интервалы по прибору и дате.
- Dry-run не изменяет базу данных.
- Ошибки доступа и операций попадают в лог.
- Повторный запуск не создаёт дублирующих броней.
- Есть минимум набор тестов: валидный конфиг, невалидный конфиг, бронирование, конфликт, просмотр занятости, ошибки.
- Есть пример запуска с cron.
- README содержит установку, конфигурацию и 2–3 примера запуска.